

users to create transactions in the Hyperledger Fabric network's shared ledger and update the world state of the assets.

Chaincode is programmable code, written in Go, and instantiated on a channel. Developers use chaincode to develop business contracts, asset definitions, and collectively-managed decentralised applications. The chaincode manages the ledger state through transactions invoked by applications. Assets are created and updated by a specific chaincode, and cannot be accessed by another chaincode.

Applications interact with the blockchain ledger through the chaincode. Therefore, the chaincode needs to be installed on every peer that will endorse a transaction and is instantiated on the channel.

There are two ways to develop smart contracts with Hyperledger Fabric:

- Code individual contracts into standalone instances of chaincode.
- A more efficient way is to use chaincode to create decentralised applications that manage the life cycle of one or multiple types of business contracts, and let the end users instantiate instances of contracts within these applications.

Overview of a chaincode program

When creating a chaincode, there are two methods that you will need to implement:

- **Init:** This is called when a chaincode receives an instantiate or upgrade transaction. This is where you will initialise any application state.
- **Invoke:** This is called when the invoke transaction is received to process any transaction proposals.

As a developer, you must create both an *Init* and an *Invoke* method within your chaincode. The chaincode must be installed using the peer chaincode install command, and instantiated using the peer chaincode instantiate command before the chaincode can be invoked. Then, transactions can be created using the peer chaincode invoke or peer chaincode query commands.

Chaincode key APIs

Important interfaces that you can use when writing your chaincode are defined by Hyperledger Fabric —ChaincodeStub and ChaincodeStubInterface. The former provides functions that allow you to interact with the underlying ledger to query, update and delete assets. The key APIs for chaincode include:

```
func (stub *ChaincodeStub) GetState(key string) ([]byte, error)
```

The above command returns the value of the specified key from the ledger. Note that *GetState* doesn't read data from the write set, which has not been committed to the ledger. In other words, *GetState* doesn't consider data modified by *PutState* that has not been committed. If the key does not exist in the state database, *(nil, nil)* is returned.

```
func (stub *ChaincodeStub) PutState(key string, value []byte) error
```

The above command puts the specified key and value into the transaction's write set as a data-write proposal. *PutState* doesn't affect the ledger until the transaction is validated and successfully committed.

```
func (stub *ChaincodeStub) DelState(key string) error
```

The above command records the specified key to be deleted in the write set of the transaction proposal. The key and its value will be deleted from the ledger when the transaction is validated and successfully committed.

With this tutorial, we complete the learning part related to Hyperledger Fabric. In my next article, we will walk through using Fabric for a specific use case. **END** 

 By: Swapnil Kulkarni

The author is experienced in blockchain, cloud native solutions, containers and enterprise software product architectures. He has diverse experience with different private and hybrid cloud architectures. He writes at cloudnativetech.wordpress.com.

OSFY Magazine Attractions During 2018-19

MONTH	THEME
March 2018	Databases management and Optimisation
April 2018	Docker and containers
May 2018	Open Source Programming (Languages and tools)
June 2018	Open Source Firewall and Network security and Monitoring
July 2018	Mobile App Development and Optimisation
August 2018	AI, Deep learning and Machine Learning
September 2018	Open Source and IoT
October 2018	BigData, Hadoop, PaaS, SaaS, IaaS and Cloud
November 2018	Open Source on Windows
December 2018	Web Development
January 2019	Data Security, Storage and Backup
February 2019	Best in the world of Open Source (Tools and Services)